

Avaliação de Programação Orientada a Objetos (POO)

Sistema de Caixa Eletrônico em Python

Objetivo

Desenvolver um **sistema de caixa eletrônico (ATM)** utilizando os principais conceitos de **Programação Orientada a Objetos**, com organização em **pacotes** e boas práticas de código.

Valor da Atividade

Total: 10,0 pontos

Forma de Entrega

- A entrega deve ser realizada **exclusivamente via GitHub**
 - Enviar apenas o **link do repositório**
 - O repositório deve conter:
 - Código-fonte organizado
 - Arquivo **README.md** explicando o projeto
 - Instruções de execução
-

Requisitos do Sistema

O sistema deve simular um caixa eletrônico com as seguintes funcionalidades:

Funcionalidades obrigatórias

- Criar conta
- Depositar dinheiro
- Sacar dinheiro
- Consultar saldo
- Listar histórico de operações

Requisitos de POO (OBRIGATÓRIOS)

Seu projeto **DEVE** implementar todos os conceitos abaixo:

1. Pacotes (1,0 ponto)

Organizar o sistema em pacotes, por exemplo:

```
banco/  
├──  
├── contas/  
├── clientes/  
├── operacoes/  
└── main.py
```

2. Classes e Objetos (1,0 ponto)

- Criar classes como:
 - `Conta`
 - `Cliente`
 - `Operacao`
 - Instanciar objetos no sistema
-

3. Herança (2,0 pontos)

- Criar diferentes tipos de contas:
 - `ContaCorrente`
 - `ContaPoupanca`
 - Ambas devem herdar de uma classe base `Conta`
-

4. Polimorfismo (2,0 pontos)

- Métodos com comportamentos diferentes nas subclasses
Exemplo:
 - Método `sacar()` com regras diferentes para cada tipo de conta
-

5. Agregação (1,5 pontos)

- Um **Banco** ou **Sistema** deve **agregar várias contas**
 - As contas podem existir independentemente do banco
-

6. Composição (1,5 pontos)

- Uma **Conta** deve ter um **Historico**
 - O histórico **não existe sem a conta**
-

7. Encapsulamento (1,0 ponto)

- Utilizar atributos privados (`_` ou `__`)
 - Criar métodos de acesso (getters/setters, quando necessário)
-

Regras de Negócio

- Não permitir saque com saldo insuficiente
 - Registrar todas as operações no histórico
 - Validar entradas do usuário
 - Exibir mensagens claras no terminal
-

Interface

- Pode ser feita via terminal (menu interativo)
- Exemplo:

1 - Criar Conta
2 - Depositar
3 - Sacar
4 - Consultar Saldo
5 - Histórico
0 - Sair

Critérios de Avaliação

Critério	Ponto s
----------	------------

Uso correto de pacotes	1,0
Classes e objetos	1,0
Herança	2,0
Polimorfismo	2,0
Agregação	1,5
Composição	1,5
Encapsulamento	1,0
Organização e boas práticas	0,5
README e documentação	0,5

Regras Importantes

-  Não copiar código da internet
 -  Não usar apenas um único arquivo
 -  Trabalhos iguais serão zerados
-



Dicas

- Modele o sistema antes de codar (pode usar UML)
 - Pense nas responsabilidades de cada classe
 - Evite código duplicado
 - Utilize nomes claros
-



Prazo

22/04/2026